

Bayesian Hierarchical Model for PM2.5

Leila Cielok

29-04-2026

Contents

1	Introduction	1
2	Libraries and data import	2
3	EDA	4
3.1	Outcome variable	4
3.2	Data	6
4	Data preprocessing	12
5	Design matrices and indexing	13
6	Prior specifications	15
7	Full conditional conditional posterior distributions specification	16
8	Metropolis-within-Gibbs sampler	17
8.1	Utility functions for the sampler	19
8.2	Postprocessing formula	20
9	Run the sampler	20
9.1	Visualization: Trace plots	20
9.2	Visualization: ACF	22
9.3	Choose burn in iterations	24
9.4	Trace plots post burn-in	24
9.5	ACF post burn-in	25
10	Posterior summaries	27
11	Sensitivity analysis	28
11.1	One-at-a-time sensitivity analysis	29
12	CODA: Geweke Diagnostics	30
13	Posterior Predictive Checks	31

1 Introduction

This project implements a Bayesian hierarchical model for daily PM2.5 concentrations observed across multiple monitoring stations. The outcome is modeled on the log scale to account for right-skewness in the raw data.

The specification is:

$$\tilde{y}_i = \log(y_i), \quad \tilde{y}_i \mid \beta, \alpha, \sigma^2 \sim \mathcal{N}(X_i^\top \beta + \alpha_{j[i]}, \sigma^2)$$

with station-specific random intercepts

$$\alpha_j \mid \tau \sim \mathcal{N}(0, \tau^2), \quad j = 1, \dots, J.$$

A Metropolis-within-Gibbs sampler is used: - Gibbs updates for β , α_j , and σ^2 - a Metropolis-Hastings step for τ

2 Libraries and data import

```
library(dplyr)
library(ggplot2)
library(readr)
library(e1071)
library(tidyr)
library(stringr)
library(purrr)
```

```
df <- read_csv("../pm25_bayes.csv", show_col_types = FALSE)
str(df)
```

```
## spc_tbl_ [31,349 x 14] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ station_id : chr [1:31349] "IT0470A" "IT0470A" "IT0470A" "IT0470A" ...
## $ Pollutant : num [1:31349] 6001 6001 6001 6001 6001 ...
## $ Start : Date[1:31349], format: "2025-01-30" "2025-01-31" ...
## $ y : num [1:31349] 29 27 5 10 26 40 45 44 35 43 ...
## $ Unit : chr [1:31349] "ug.m-3" "ug.m-3" "ug.m-3" "ug.m-3" ...
## $ Country : chr [1:31349] "Italy" "Italy" "Italy" "Italy" ...
## $ Municipality: chr [1:31349] "Piazza Rebaudengo 23" "Piazza Rebaudengo 23" "Piazza Rebaudengo 23" ...
## $ Longitude : num [1:31349] 7.7 7.7 7.7 7.7 7.7 ...
## $ Latitude : num [1:31349] 45.1 45.1 45.1 45.1 45.1 ...
## $ Altitude : num [1:31349] 233 233 233 233 233 233 233 233 233 233 ...
## $ month : chr [1:31349] "2025-01" "2025-01" "2025-02" "2025-02" ...
## $ day_of_week : num [1:31349] 3 4 5 6 0 1 2 3 4 1 ...
## $ station_idx : num [1:31349] 0 0 0 0 0 0 0 0 0 0 ...
## $ date : Date[1:31349], format: "2025-01-30" "2025-01-31" ...
## - attr(*, "spec")=
## .. cols(
## .. station_id = col_character(),
## .. Pollutant = col_double(),
## .. Start = col_date(format = ""),
## .. y = col_double(),
## .. Unit = col_character(),
## .. Country = col_character(),
## .. Municipality = col_character(),
## .. Longitude = col_double(),
## .. Latitude = col_double(),
## .. Altitude = col_double(),
## .. month = col_character(),
```

```
## .. day_of_week = col_double(),
## .. station_idx = col_double(),
## .. date = col_date(format = "")
## .. )
## - attr(*, "problems")=<externalptr>
```

```
df$Pollutant <- NULL
df <- df %>%
  mutate(
    Start = as.Date(Start),
    month = factor(month),
    day_of_week = factor(
      day_of_week,
      levels = 0:6,
      labels = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
    )
  )
summary(df)
```

```
## station_id          Start          y          Unit
## Length:31349      Min.   :2025-01-30      Min.   : 0.01      Length:31349
## Class :character  1st Qu.:2025-04-21      1st Qu.: 6.00      Class :character
## Mode  :character  Median :2025-07-13      Median : 9.00      Mode  :character
##                  Mean   :2025-07-14      Mean   :11.33
##                  3rd Qu.:2025-10-08      3rd Qu.:13.40
##                  Max.   :2025-12-30      Max.   :775.60
##
## Country           Municipality           Longitude           Latitude
## Length:31349      Length:31349           Min.   : 7.538      Min.   :35.50
## Class :character  Class :character  1st Qu.:11.802      1st Qu.:39.07
## Mode  :character  Mode  :character  Median :12.566      Median :42.10
##                  Mean   :12.339      Mean   :41.86
##                  3rd Qu.:13.533      3rd Qu.:43.46
##                  Max.   :17.114      Max.   :46.13
##
## Altitude          month          day_of_week  station_idx
## Min.   : 0.0      2025-03: 3091      Mon:4485      Min.   : 0.00
## 1st Qu.: 35.0      2025-04: 2914      Tue:4447      1st Qu.: 25.00
## Median :160.0      2025-10: 2908      Wed:4415      Median : 56.00
## Mean   :225.2      2025-05: 2900      Thu:4528      Mean   : 55.65
## 3rd Qu.:338.0      2025-11: 2852      Fri:4513      3rd Qu.: 83.00
## Max.   :1038.0      2025-08: 2844      Sat:4471      Max.   :115.00
##                  (Other):13840      Sun:4490
##
## date
## Min.   :2025-01-30
## 1st Qu.:2025-04-21
## Median :2025-07-13
## Mean   :2025-07-14
## 3rd Qu.:2025-10-08
## Max.   :2025-12-30
##
```

3 EDA

3.1 Outcome variable

```
# Basic summary statistics
outcome_var <- "y"

summary(df[[outcome_var]])

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.01   6.00   9.00  11.33  13.40  775.60

# Missing and non-positive values
cat("Number of missing values:", sum(is.na(df[[outcome_var]])), "\n")

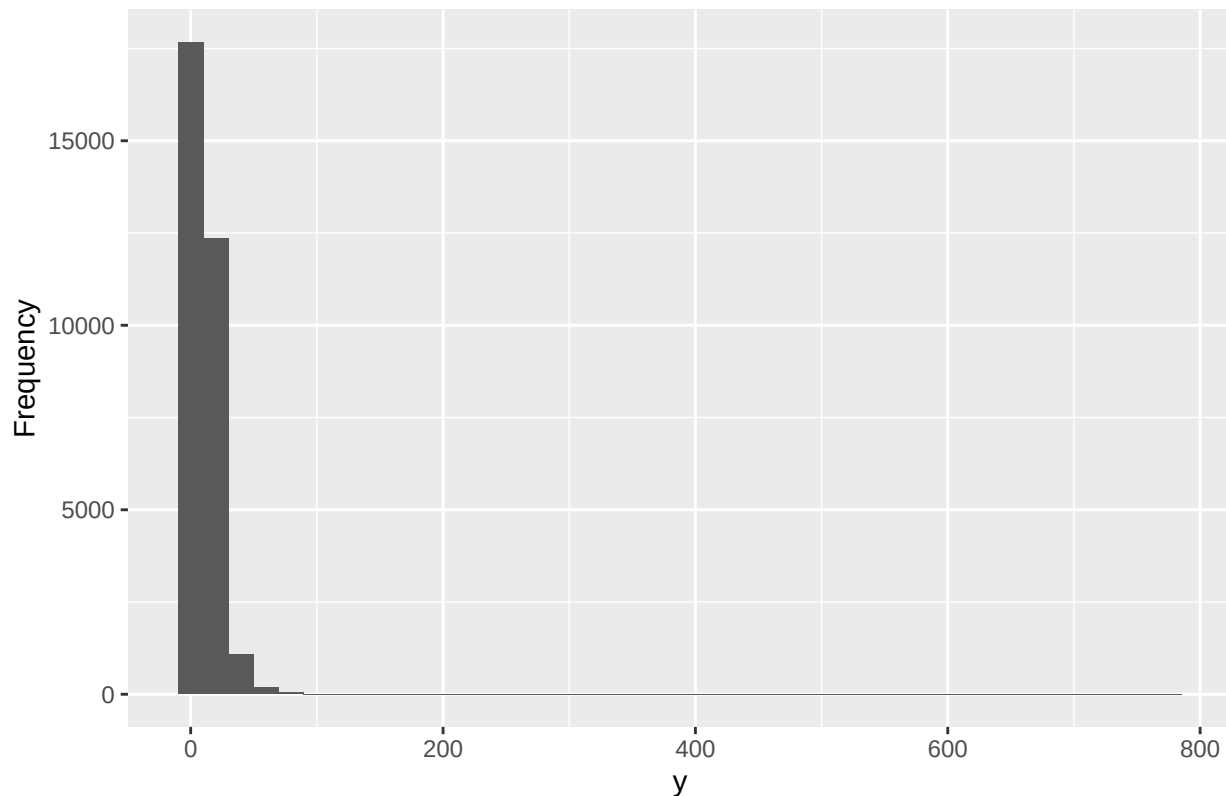
## Number of missing values: 0

cat("Number of non-positive values:", sum(df[[outcome_var]] <= 0, na.rm = TRUE), "\n")

## Number of non-positive values: 0

# Histogram of raw outcome
ggplot(df, aes(x = .data[[outcome_var]])) +
  geom_histogram(bins = 40) +
  labs(
    title = "Distribution of the raw PM2.5 outcome",
    x = outcome_var,
    y = "Frequency"
  )
```

Distribution of the raw PM2.5 outcome



```

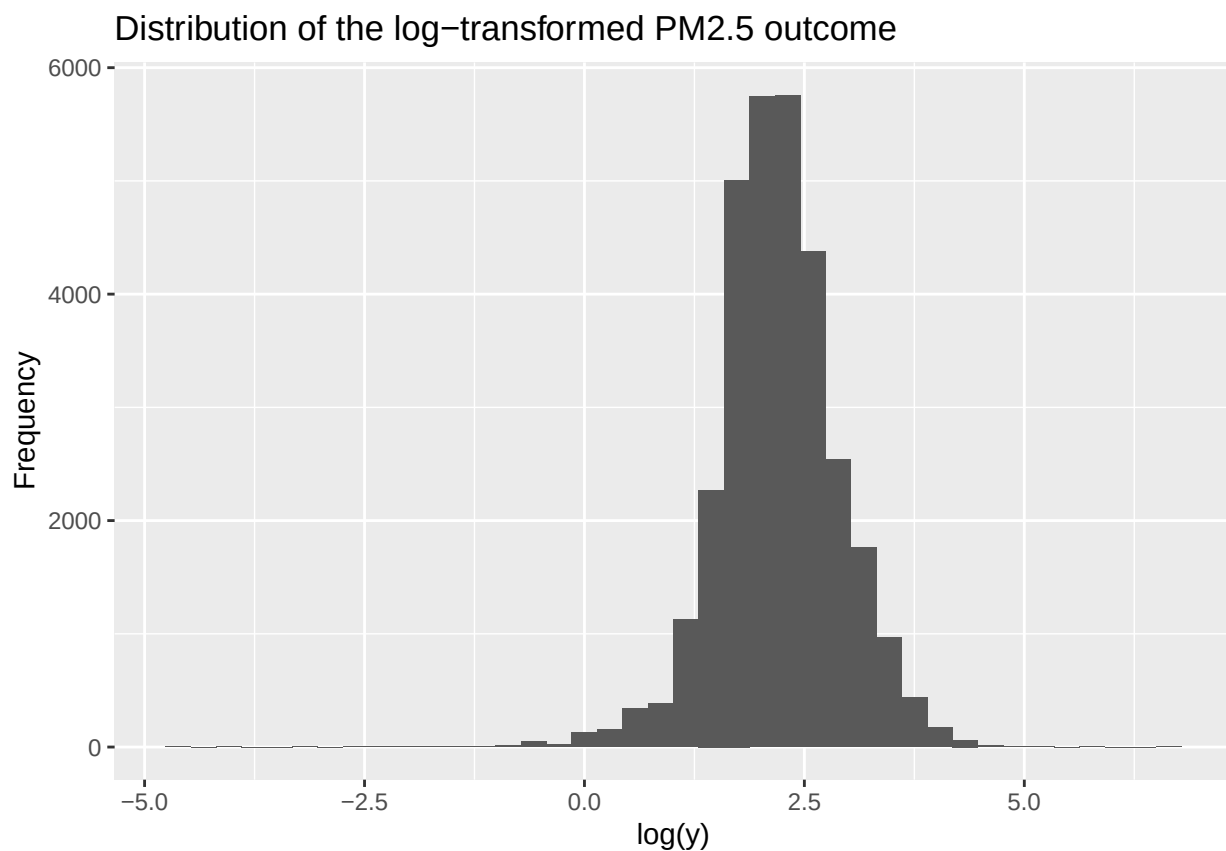
# Skewness of raw data
skew_raw <- skewness(df[[outcome_var]], na.rm = TRUE)
cat("Skewness (raw):", skew_raw, "\n")

## Skewness (raw): 17.05869

# Histogram of log-transformed outcome
df_pos <- df %>% filter(.data[[outcome_var]] > 0)

ggplot(df_pos, aes(x = log(.data[[outcome_var]]))) +
  geom_histogram(bins = 40) +
  labs(
    title = "Distribution of the log-transformed PM2.5 outcome",
    x = paste0("log(", outcome_var, ")"),
    y = "Frequency"
  )

```



```

# Skewness of log data
skew_log <- skewness(log(df[[outcome_var]]), na.rm = TRUE)
cat("Skewness (log):", skew_log, "\n")

## Skewness (log): -0.2855041

# Compare raw and log-scale summaries
cat("\nSummary of raw outcome:\n")

##
## Summary of raw outcome:

```

```
print(summary(df_pos[[outcome_var]]))

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.01   6.00   9.00   11.33   13.40   775.60

cat("\nSummary of log-transformed outcome:\n")

##
## Summary of log-transformed outcome:
print(summary(log(df_pos[[outcome_var]])))

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     -4.605   1.792   2.197   2.193   2.595   6.654
```

3.2 Data

```
# 1. Observations per station
obs_per_station <- df %>%
  count(station_id, name = "n_obs") %>%
  arrange(desc(n_obs))

cat("Number of stations:", n_distinct(df$station_id), "\n")

## Number of stations: 116

cat("Min observations per station:", min(obs_per_station$n_obs), "\n")

## Min observations per station: 69

cat("Median observations per station:", median(obs_per_station$n_obs), "\n")

## Median observations per station: 299.5

cat("Max observations per station:", max(obs_per_station$n_obs), "\n")

## Max observations per station: 332

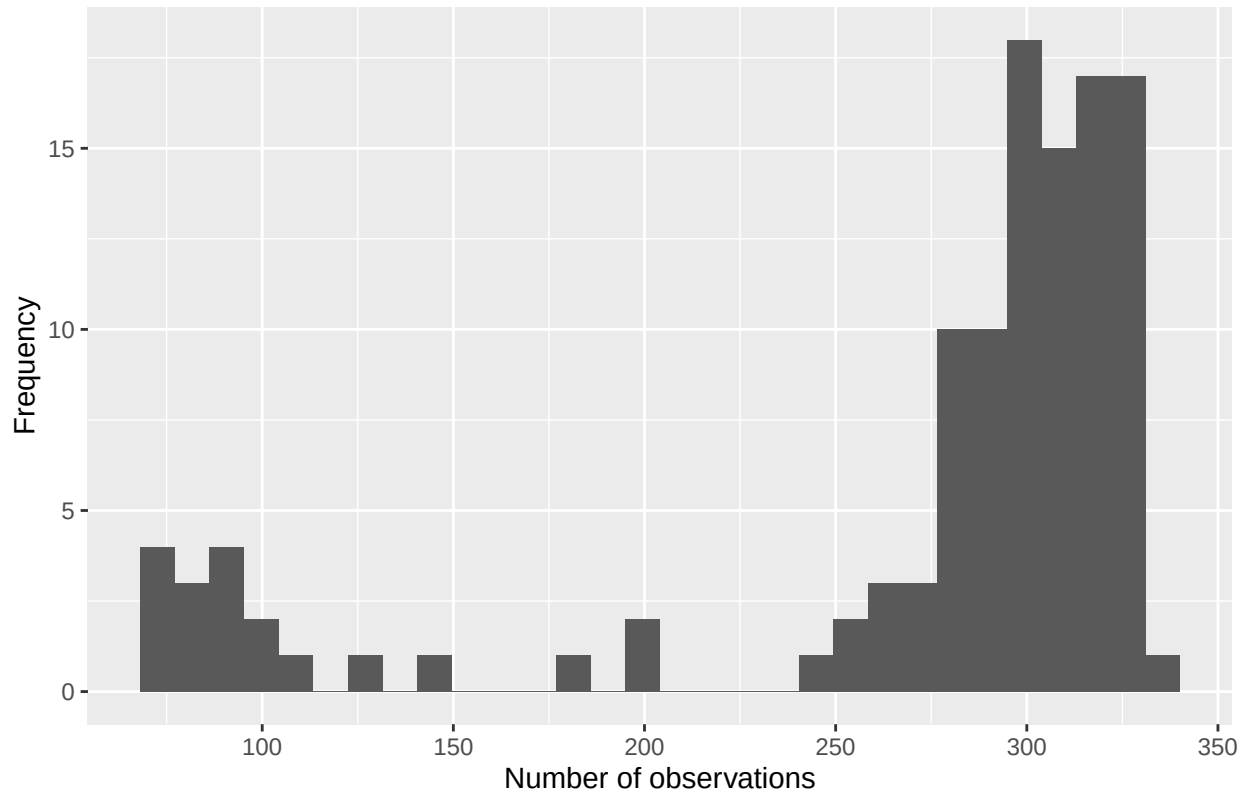
head(obs_per_station, 10)

## # A tibble: 10 x 2
##   station_id n_obs
##   <chr>      <int>
## 1 IT2254A    332
## 2 IT1906A    330
## 3 IT2023A    329
## 4 IT0863A    328
## 5 IT0889A    327
## 6 IT0956A    326
## 7 IT0825A    325
## 8 IT1836A    325
## 9 IT2174A    325
## 10 IT2247A    325

ggplot(obs_per_station, aes(x = n_obs)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Distribution of observations per station",
```

```
x = "Number of observations",
y = "Frequency"
)
```

Distribution of observations per station

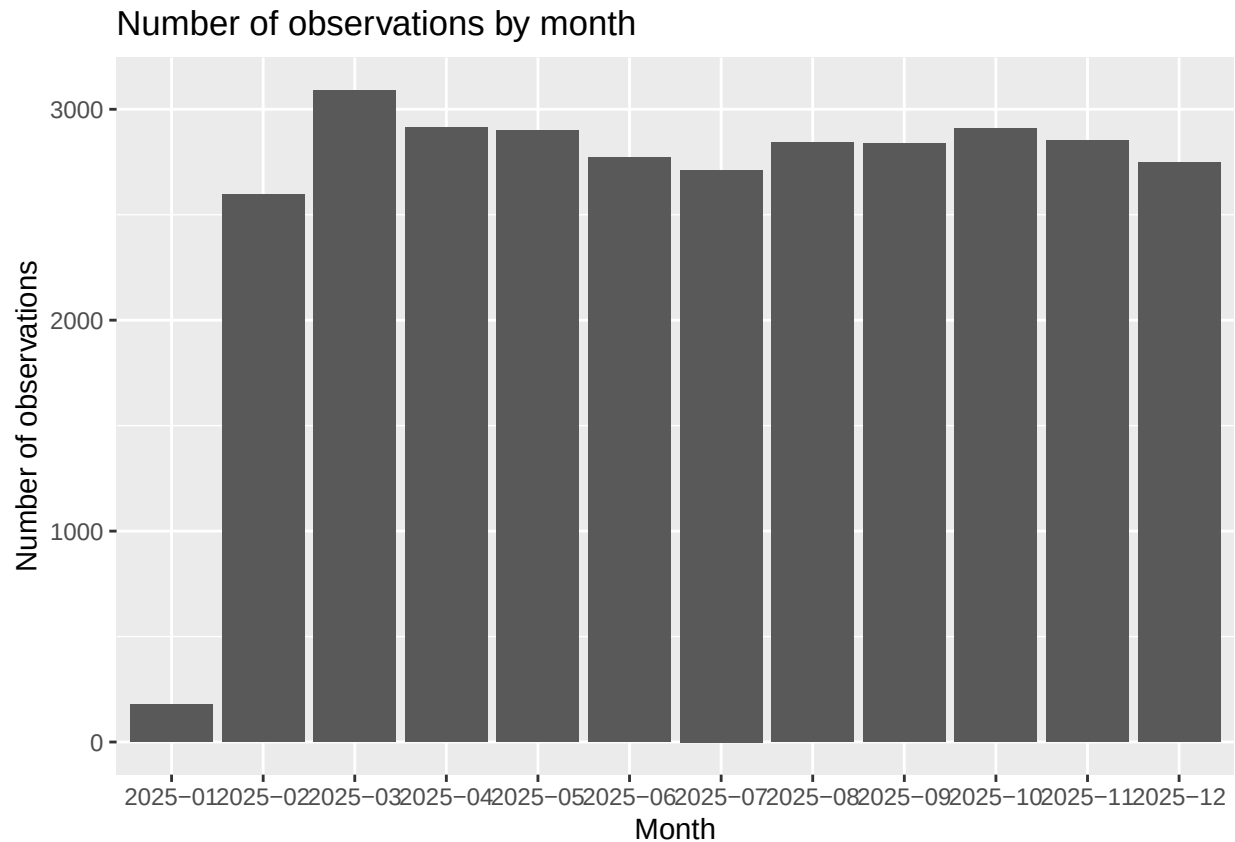


```
# 2a Observations per month
obs_per_month <- df %>%
  count(month, name = "n_obs") %>%
  arrange(month)

print(obs_per_month)
```

```
## # A tibble: 12 x 2
##   month  n_obs
##   <fct> <int>
## 1 2025-01  177
## 2 2025-02 2594
## 3 2025-03 3091
## 4 2025-04 2914
## 5 2025-05 2900
## 6 2025-06 2770
## 7 2025-07 2712
## 8 2025-08 2844
## 9 2025-09 2839
## 10 2025-10 2908
## 11 2025-11 2852
## 12 2025-12 2748
```

```
ggplot(obs_per_month, aes(x = factor(month), y = n_obs)) +
  geom_col() +
  labs(
    title = "Number of observations by month",
    x = "Month",
    y = "Number of observations"
  )
```



```
# 2b Observations per day of week
obs_per_day <- df %>%
  count(day_of_week, name = "n_obs")

print(obs_per_day)
```

```
## # A tibble: 7 x 2
##   day_of_week n_obs
##   <fct>      <int>
## 1 Mon         4485
## 2 Tue         4447
## 3 Wed         4415
## 4 Thu         4528
## 5 Fri         4513
## 6 Sat         4471
## 7 Sun         4490
```

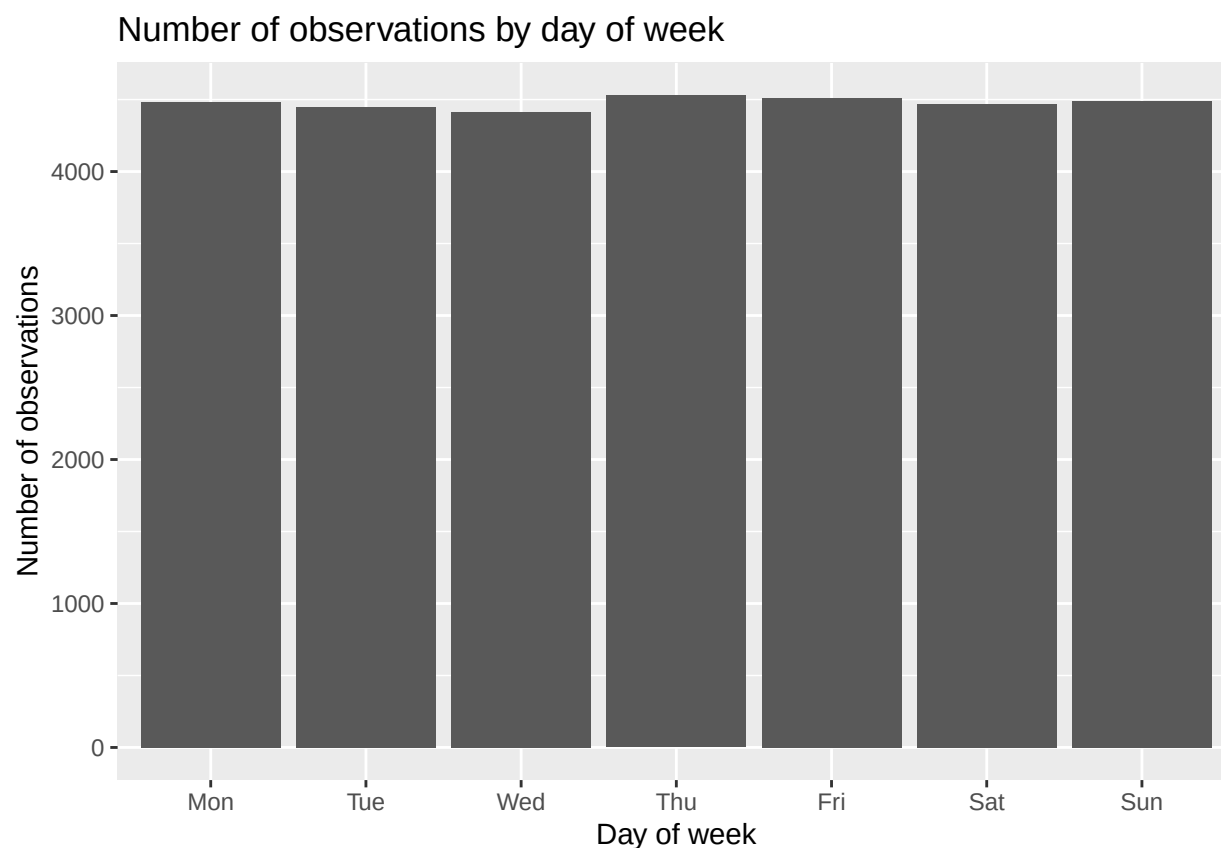
```
ggplot(obs_per_day, aes(x = day_of_week, y = n_obs)) +
  geom_col() +
  labs(
```



```

title = "Number of observations by day of week",
x = "Day of week",
y = "Number of observations"
)

```



```

# 3. Station-month panel structure
station_month <- df %>%
  distinct(station_id, month) %>%
  count(station_id, name = "n_months_present") %>%
  arrange(n_months_present, station_id)

cat("Min number of months observed per station:", min(station_month$n_months_present), "\n")

## Min number of months observed per station: 4

cat("Median number of months observed per station:", median(station_month$n_months_present), "\n")

## Median number of months observed per station: 12

cat("Max number of months observed per station:", max(station_month$n_months_present), "\n")

## Max number of months observed per station: 12

head(station_month, 10)

## # A tibble: 10 x 2
##   station_id n_months_present
##   <chr>      <int>
## 1 IT1830A      4

```

```

## 2 IT1948A 5
## 3 IT1788A 7
## 4 IT2313A 7
## 5 IT1740A 8
## 6 IT1899A 8
## 7 IT2202A 8
## 8 IT2203A 8
## 9 IT2204A 8
## 10 IT2205A 8

# 4. Full station x month table and missing months
station_month_full <- expand_grid(
  station_id = sort(unique(df$station_id)),
  month = sort(unique(df$month))
) %>%
  left_join(
    df %>%
      count(station_id, month, name = "n_obs"),
    by = c("station_id", "month")
  ) %>%
  mutate(
    n_obs = ifelse(is.na(n_obs), 0, n_obs),
    missing_month = ifelse(n_obs == 0, 1, 0)
  )

missing_months_by_station <- station_month_full %>%
  group_by(station_id) %>%
  summarise(
    missing_months = sum(missing_month),
    months_observed = sum(n_obs > 0),
    .groups = "drop"
  ) %>%
  arrange(desc(missing_months), station_id)

cat("Stations with at least one completely missing month:",
    sum(missing_months_by_station$missing_months > 0), "\n")

## Stations with at least one completely missing month: 32

head(missing_months_by_station, 15)

## # A tibble: 15 x 3
##   station_id missing_months months_observed
##   <chr>          <dbl>          <int>
## 1 IT1830A          8            4
## 2 IT1948A          7            5
## 3 IT1788A          5            7
## 4 IT2313A          5            7
## 5 IT1740A          4            8
## 6 IT1899A          4            8
## 7 IT2202A          4            8
## 8 IT2203A          4            8
## 9 IT2204A          4            8
## 10 IT2205A         4            8
## 11 IT1247A         3            9

```

```
## 12 IT1487A          3          9
## 13 IT2211A          3          9
## 14 IT2268A          3          9
## 15 IT2277A          3          9
```

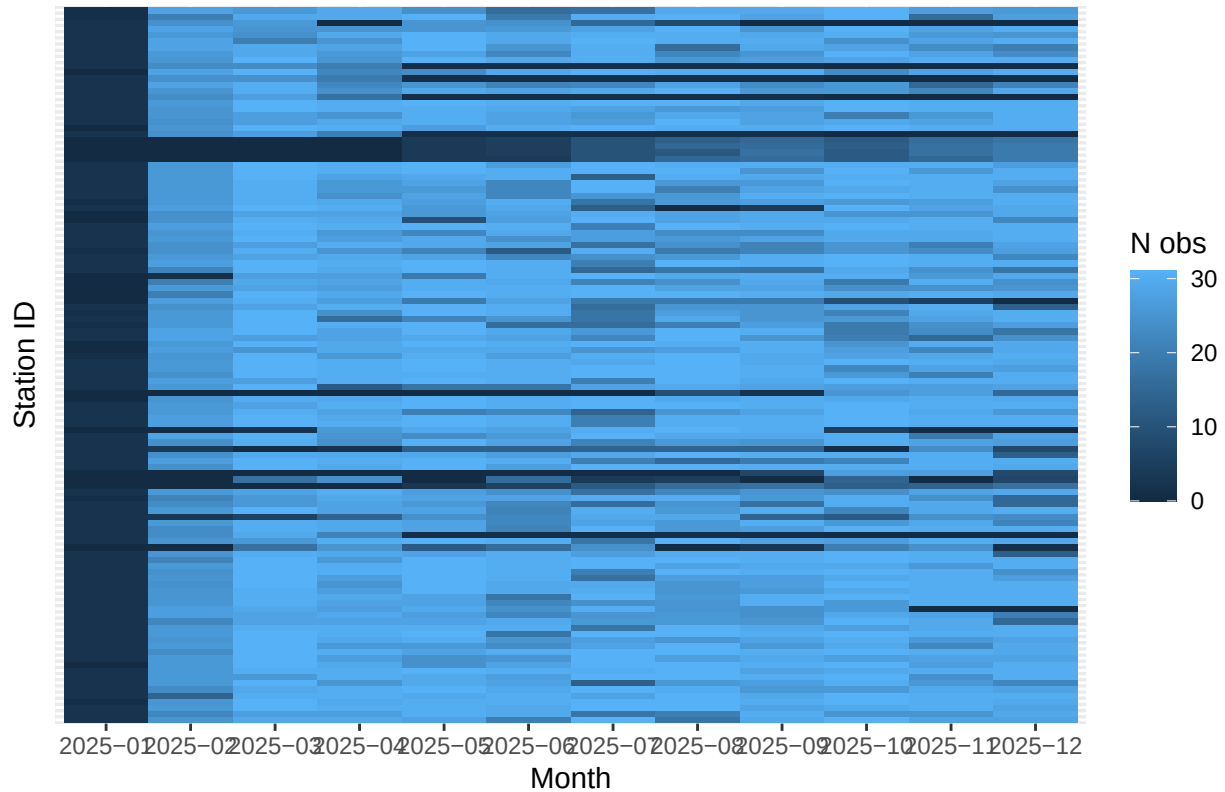
```
# Months missing for each station
missing_month_detail <- station_month_full %>%
  filter(n_obs == 0) %>%
  arrange(station_id, month)

head(missing_month_detail, 20)
```

```
## # A tibble: 20 x 4
##   station_id month   n_obs missing_month
##   <chr>      <fct>   <dbl>      <dbl>
## 1 IT0952A    2025-01     0          1
## 2 IT1079A    2025-11     0          1
## 3 IT1079A    2025-12     0          1
## 4 IT1247A    2025-01     0          1
## 5 IT1247A    2025-02     0          1
## 6 IT1247A    2025-08     0          1
## 7 IT1487A    2025-10     0          1
## 8 IT1487A    2025-11     0          1
## 9 IT1487A    2025-12     0          1
## 10 IT1740A   2025-01     0          1
## 11 IT1740A   2025-02     0          1
## 12 IT1740A   2025-03     0          1
## 13 IT1740A   2025-04     0          1
## 14 IT1788A   2025-01     0          1
## 15 IT1788A   2025-02     0          1
## 16 IT1788A   2025-05     0          1
## 17 IT1788A   2025-09     0          1
## 18 IT1788A   2025-11     0          1
## 19 IT1830A   2025-01     0          1
## 20 IT1830A   2025-02     0          1
```

```
# Heatmap of data availability by station and month
ggplot(station_month_full, aes(x = factor(month), y = factor(station_id), fill = n_obs)) +
  geom_tile() +
  labs(
    title = "Data availability by station and month",
    x = "Month",
    y = "Station ID",
    fill = "N obs"
  ) +
  theme(axis.text.y = element_blank(),
        axis.ticks.y = element_blank())
```

Data availability by station and month



Exploratory analysis shows that the raw PM2.5 outcome is highly right-skewed, with a skewness of 17.06 and a long upper tail driven by extreme observations. After log transformation, skewness drops to -0.29, yielding a much more symmetric distribution and supporting the use of a Gaussian likelihood for the transformed outcome.

The panel structure is markedly unbalanced across monitoring stations. The number of observations per station ranges from 69 to 332, with a median of 299.5. The dataset is also uneven across months: while most months contain between roughly 2,600 and 3,100 observations, January is severely underrepresented, with only 177 observations. Moreover, 32 stations exhibit at least one completely missing month, and the minimum temporal coverage is only 4 months for some stations. These features support the use of a hierarchical specification, which is well suited to unbalanced grouped data.

4 Data preprocessing

```
# Create log outcome and keep only finite values
df <- df %>%
  mutate(log_y = log(.data[[outcome_var]])) %>%
  filter(is.finite(log_y))

# Keep only variables used in the model and clean them
model_vars <- c("log_y", "station_id", "month", "day_of_week")

df_model <- df %>%
  dplyr::select(all_of(model_vars)) %>%
  drop_na() %>%
  mutate(
    station_id = factor(station_id),
```

```

    month = factor(month),
    day_of_week = factor(day_of_week)
  )

cat("Number of observations:", nrow(df_model), "\n")

## Number of observations: 31349

cat("Number of stations:", n_distinct(df_model$station_id), "\n")

## Number of stations: 116

# Summary of transformed outcome
df_model %>%
  summarise(
    n = n(),
    mean_log_y = mean(log_y),
    sd_log_y = sd(log_y),
    min_log_y = min(log_y),
    max_log_y = max(log_y)
  )

## # A tibble: 1 x 5
##       n mean_log_y sd_log_y min_log_y max_log_y
##   <int>      <dbl>   <dbl>    <dbl>    <dbl>
## 1 31349        2.19   0.686    -4.61     6.65

```

5 Design matrices and indexing

```

# Fixed-effects matrix
X <- model.matrix(~ month + day_of_week, data = df_model)

# Outcome vector
y_tilde <- df_model$log_y

# Group index for stations
station_index <- as.integer(df_model$station_id)

# Dimensions
J <- nlevels(df_model$station_id)
n <- length(y_tilde)
p <- ncol(X)

# Number of observations per group
n_j <- as.integer(table(df_model$station_id))

cat("n =", n, "\n")

## n = 31349

cat("p =", p, "\n")

## p = 18

cat("J =", J, "\n")

```

```
## J = 116
```

```
# check  
head(X)
```

```
##      (Intercept) month2025-02 month2025-03 month2025-04 month2025-05 month2025-06  
## 1             1             0             0             0             0             0  
## 2             1             0             0             0             0             0  
## 3             1             1             0             0             0             0  
## 4             1             1             0             0             0             0  
## 5             1             1             0             0             0             0  
## 6             1             1             0             0             0             0  
##      month2025-07 month2025-08 month2025-09 month2025-10 month2025-11 month2025-12  
## 1             0             0             0             0             0             0  
## 2             0             0             0             0             0             0  
## 3             0             0             0             0             0             0  
## 4             0             0             0             0             0             0  
## 5             0             0             0             0             0             0  
## 6             0             0             0             0             0             0  
##      day_of_weekTue day_of_weekWed day_of_weekThu day_of_weekFri day_of_weekSat  
## 1             0             0             1             0             0  
## 2             0             0             0             1             0  
## 3             0             0             0             0             1  
## 4             0             0             0             0             0  
## 5             0             0             0             0             0  
## 6             1             0             0             0             0  
##      day_of_weekSun  
## 1             0  
## 2             0  
## 3             0  
## 4             1  
## 5             0  
## 6             0
```

```
table(df_model$month)
```

```
##  
## 2025-01 2025-02 2025-03 2025-04 2025-05 2025-06 2025-07 2025-08 2025-09 2025-10  
##    177    2594    3091    2914    2900    2770    2712    2844    2839    2908  
## 2025-11 2025-12  
##    2852    2748
```

```
table(df_model$day_of_week)
```

```
##  
## Mon Tue Wed Thu Fri Sat Sun  
## 4485 4447 4415 4528 4513 4471 4490
```

```
summary(df_model$station_id)
```

```
## IT2254A IT1906A IT2023A IT0863A IT0889A IT0956A IT0825A IT1836A IT2174A IT2247A  
##    332    330    329    328    327    326    325    325    325    325  
## IT1184A IT2172A IT1183A IT1940A IT2100A IT2109A IT1900A IT2031A IT0992A IT1902A  
##    324    324    323    323    323    323    322    322    321    321  
## IT2087A IT2013A IT2252A IT0952A IT2004A IT0887A IT2026A IT2012A IT0867A IT0871A  
##    321    319    319    318    318    317    317    315    314    314  
## IT1143A IT1144A IT1182A IT2286A IT2249A IT1011A IT2308A IT1130A IT1510A IT2107A
```

##	314	314	314	314	313	312	311	310	308	308
##	IT2134A	IT2307A	IT1365A	IT0989A	IT2128A	IT0953A	IT1222A	IT2281A	IT2117A	IT2312A
##	308	308	307	306	306	305	305	305	304	304
##	IT1180A	IT1532A	IT1871A	IT1890A	IT2089A	IT2274A	IT1128A	IT2111A	IT0957A	IT2028A
##	303	303	303	301	301	301	300	300	299	299
##	IT0470A	IT2101A	IT1523A	IT2114A	IT1883A	IT2052A	IT2251A	IT2108A	IT2118A	IT2067A
##	298	298	297	297	296	296	296	295	294	293
##	IT2294A	IT2317A	IT1065A	IT2297A	IT1596A	IT2110A	IT1078A	IT1860A	IT0885A	IT0614A
##	292	292	289	289	288	288	287	286	285	284
##	IT2093A	IT2318A	IT1728A	IT2068A	IT1904A	IT2033A	IT2094A	IT2066A	IT2105A	IT1997A
##	283	283	282	282	281	279	279	277	275	274
##	IT2064A	IT2099A	IT2275A	IT1590A	IT2104A	IT2113A	IT1079A	IT1529A	IT2078A	(Other)
##	268	265	263	259	258	250	243	202	202	1648

6 Prior specifications

Weakly informative priors are adopted for all model parameters, reflecting limited prior knowledge while ensuring posterior regularization.

For the regression coefficients, a multivariate normal prior is assumed:

$$\beta \sim \mathcal{N}(b_0, B_0),$$

where b_0 is a prior mean vector and $B_0 = cI$ is a diagonal covariance matrix with c controlling the prior variance.

For the station-specific random effects, I specify:

$$\alpha_j \mid \tau \sim \mathcal{N}(0, \tau^2), \quad j = 1, \dots, J,$$

which induces partial pooling across monitoring stations.

For the observation noise variance, an inverse-gamma prior is assumed:

$$\sigma^2 \sim \text{IG}(a_\sigma, b_\sigma),$$

with small values of a_σ and b_σ to obtain a weakly informative specification.

Finally, for the scale parameter of the random effects, a half-Cauchy prior is used:

$$\tau \sim C^+(0, A),$$

which is commonly recommended for hierarchical variance components due to its heavy tails and robustness properties.

These choices are motivated by both modeling and computational considerations. In particular, conjugate priors are adopted for β , α_j , and σ^2 to allow for efficient Gibbs updates, while a non-conjugate prior is specified for τ , requiring a Metropolis–Hastings step. Overall, the specification ensures sufficient flexibility while preventing overfitting, and will be further assessed through a prior sensitivity analysis.

```
# -----
# Prior hyperparameters
# -----
```

```

# beta ~ N(b0, B0)
c_beta <- 100
b0 <- rep(0, p)
B0 <- diag(c_beta, p)
B0_inv <- solve(B0)

# sigma2 ~ IG(a_sigma, b_sigma)
a_sigma <- 2
b_sigma <- 0.5

# tau ~ Half-Cauchy(0, A)
A_tau <- 5

# -----
# MCMC settings
# -----
n_iter <- 5000
burn_in <- 1000

# proposal sd for log(tau)
proposal_sd_tau <- 0.15

# -----
# Initial values
# -----
beta_curr <- rep(0, p)
alpha_curr <- rep(0, J)
sigma2_curr <- var(y_tilde)
tau_curr <- 1

# -----
# Storage
# -----
beta_store <- matrix(NA_real_, nrow = n_iter, ncol = p)
alpha_store <- matrix(NA_real_, nrow = n_iter, ncol = J)
sigma2_store <- numeric(n_iter)
tau_store <- numeric(n_iter)
accept_tau <- logical(n_iter)

```

7 Full conditional conditional posterior distributions specification

The full conditional posterior distributions are derived and passed to the Gibbs sampling algorithm:

$$\beta \mid \cdot \sim \mathcal{N}(m_\beta, V_\beta)$$

with

$$V_\beta = \left(B_0^{-1} + \frac{1}{\sigma^2} X^\top X \right)^{-1}, \quad m_\beta = V_\beta \left(B_0^{-1} b_0 + \frac{1}{\sigma^2} X^\top (\tilde{y} - Z\alpha) \right)$$

$$\alpha_j \mid \cdot \sim \mathcal{N}(m_j, V_j)$$

with

$$V_j = \left(\frac{n_j}{\sigma^2} + \frac{1}{\tau^2} \right)^{-1}, \quad m_j = V_j \frac{1}{\sigma^2} \sum_{i \in j} (\tilde{y}_i - x_i^\top \beta)$$

$$\sigma^2 \mid \cdot \sim \text{IG} \left(a_\sigma + \frac{n}{2}, b_\sigma + \frac{SSR}{2} \right)$$

where

$$SSR = \sum_{i=1}^n (\tilde{y}_i - x_i^\top \beta - \alpha_{j[i]})^2.$$

The full conditional distribution of τ is not available in closed form. Up to proportionality, it is given by:

$$p(\tau \mid \alpha) \propto \left[\prod_{j=1}^J \mathcal{N}(\alpha_j \mid 0, \tau^2) \right] \cdot p(\tau)$$

Assuming a half-Cauchy prior $\tau \sim C^+(0, A)$, this yields:

$$p(\tau \mid \alpha) \propto \tau^{-J} \exp \left(-\frac{1}{2\tau^2} \sum_{j=1}^J \alpha_j^2 \right) \cdot \frac{1}{1 + (\tau/A)^2}$$

This distribution does not belong to a standard family, hence we use a Metropolis–Hastings step within the Gibbs sampler.

8 Metropolis-within-Gibbs sampler

```
run_mwg_sampler <- function(y_tilde, X, station_index, n_j,
                             b0, B0, a_sigma, b_sigma, A_tau,
                             n_iter = 5000,
                             proposal_sd = 0.15,
                             seed = 123) {

  set.seed(seed)

  n <- length(y_tilde)
  p <- ncol(X)
  J <- length(n_j)
  B0_inv <- solve(B0)

  if (any(is.na(station_index))) stop("station_index contains NA")
  if (min(station_index) < 1) stop("station_index contains values < 1")
  if (max(station_index) > J) stop("station_index exceeds number of groups")
  if (burn_in >= n_iter) stop("burn_in must be smaller than n_iter")

  # Storage
  beta_store <- matrix(NA_real_, nrow = n_iter, ncol = p)
  alpha_store <- matrix(NA_real_, nrow = n_iter, ncol = J)
  sigma2_store <- numeric(n_iter)
```

```

tau_store <- numeric(n_iter)
accept_tau <- logical(n_iter)

# Initial values
beta_curr <- rep(0, p)
alpha_curr <- rep(0, J)
sigma2_curr <- var(y_tilde)
tau_curr <- 1

XtX <- crossprod(X)

for (t in seq_len(n_iter)) {

  # ---- 1. Gibbs update for beta ----
  alpha_vec <- alpha_curr[station_index]

  precision_beta <- B0_inv + XtX / sigma2_curr
  V_beta <- solve(precision_beta)
  V_beta <- 0.5 * (V_beta + t(V_beta))

  term_prior <- B0_inv %*% b0
  term_lik <- crossprod(X, y_tilde - alpha_vec) / sigma2_curr
  rhs_beta <- term_prior + term_lik
  m_beta <- solve(precision_beta, rhs_beta)

  if (any(!is.finite(m_beta))) {
    stop(paste("m_beta contains non-finite values at iteration", t))
  }

  beta_curr <- as.numeric(MASS::mvrnorm(
    1,
    mu = as.numeric(m_beta),
    Sigma = V_beta
  ))

  # ---- 2. Gibbs updates for alpha_j ----
  xb <- as.numeric(X %*% beta_curr)

  for (j in seq_len(J)) {
    idx <- which(station_index == j)
    V_j <- 1 / (n_j[j] / sigma2_curr + 1 / (tau_curr^2))
    m_j <- V_j * sum(y_tilde[idx] - xb[idx]) / sigma2_curr
    alpha_curr[j] <- rnorm(1, mean = m_j, sd = sqrt(V_j))
  }

  # ---- 3. Gibbs update for sigma^2 ----
  resid <- y_tilde - xb - alpha_curr[station_index]
  SSR <- sum(resid^2)
  sigma2_curr <- rinvgamma(
    1,
    shape = a_sigma + n / 2,
    scale = b_sigma + SSR / 2
  )
}

```

```

if (!is.finite(sigma2_curr) || sigma2_curr <= 0) {
  stop(paste("sigma2_curr became invalid at iteration", t))
}

# ---- 4. MH update for tau on log scale ----
eta_curr <- log(tau_curr)
eta_prop <- rnorm(1, mean = eta_curr, sd = proposal_sd)

log_r <- log_target_eta(eta_prop, alpha_curr, A_tau) -
  log_target_eta(eta_curr, alpha_curr, A_tau)

if (log(runif(1)) < log_r) {
  tau_curr <- exp(eta_prop)
  accept_tau[t] <- TRUE
} else {
  accept_tau[t] <- FALSE
}

# ---- Store current draws ----
beta_store[t, ] <- beta_curr
alpha_store[t, ] <- alpha_curr
sigma2_store[t] <- sigma2_curr
tau_store[t] <- tau_curr
}

list(
  beta = beta_store,
  alpha = alpha_store,
  sigma2 = sigma2_store,
  tau = tau_store,
  accept_tau = accept_tau,
  accept_rate_tau_all = mean(accept_tau)
)
}

```

8.1 Utility functions for the sampler

```

# Draw from Inverse-Gamma(shape, scale) under the convention:
#  $p(x)$  prop to  $x^{-(a+1)} \exp(-b/x)$ 
# If  $G \sim \text{Gamma}(\text{shape}=a, \text{rate}=b)$ , then  $1/G \sim \text{IG}(a, b)$ 
rinvgamma <- function(n, shape, scale) {
  1 / rgamma(n, shape = shape, rate = scale)
}

# Log-target for eta = log(tau)
log_target_eta <- function(eta, alpha, A_tau) {
  J <- length(alpha)
  S_alpha <- sum(alpha^2)
  -(J - 1) * eta - S_alpha / (2 * exp(2 * eta)) - log(1 + exp(2 * eta) / A_tau^2)
}

```

8.2 Postprocessing formula

```
post_process_mcmc <- function(mcmc_fit, burn_in = 100, thin = 1) {
  n_iter <- nrow(mcmc_fit$beta)
  if (burn_in >= n_iter) stop("burn_in must be smaller than total iterations")
  if (thin < 1) stop("thin must be >= 1")

  kept <- seq(from = burn_in + 1, to = n_iter, by = thin)

  list(
    post_beta = mcmc_fit$beta[kept, , drop = FALSE],
    post_alpha = mcmc_fit$alpha[kept, , drop = FALSE],
    post_sigma2 = mcmc_fit$sigma2[kept],
    post_tau = mcmc_fit$tau[kept],
    accept_rate_tau = mean(mcmc_fit$accept_tau[kept]),
    kept_iterations = kept
  )
}
```

9 Run the sampler

```
# MASS::mvrnorm is used inside the function
library(MASS)
```

```
mcmc_fit <- run_mwg_sampler(
  y_tilde = y_tilde,
  X = X,
  station_index = station_index,
  n_j = n_j,
  b0 = b0,
  B0 = B0,
  a_sigma = a_sigma,
  b_sigma = b_sigma,
  A_tau = A_tau,
  n_iter = 5000,
  proposal_sd = 0.2,
  seed = 123
)
```

```
mcmc_fit$accept_rate_tau_all
```

```
## [1] 0.3782
```

9.1 Visualization: Trace plots

```
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))

plot(mcmc_fit$beta[, 1], type = "l",
     main = expression(paste("Trace plot of ", beta[1])),
     xlab = "Iteration", ylab = "Value")

plot(mcmc_fit$beta[, 2], type = "l",
     main = expression(paste("Trace plot of ", beta[2])),
```

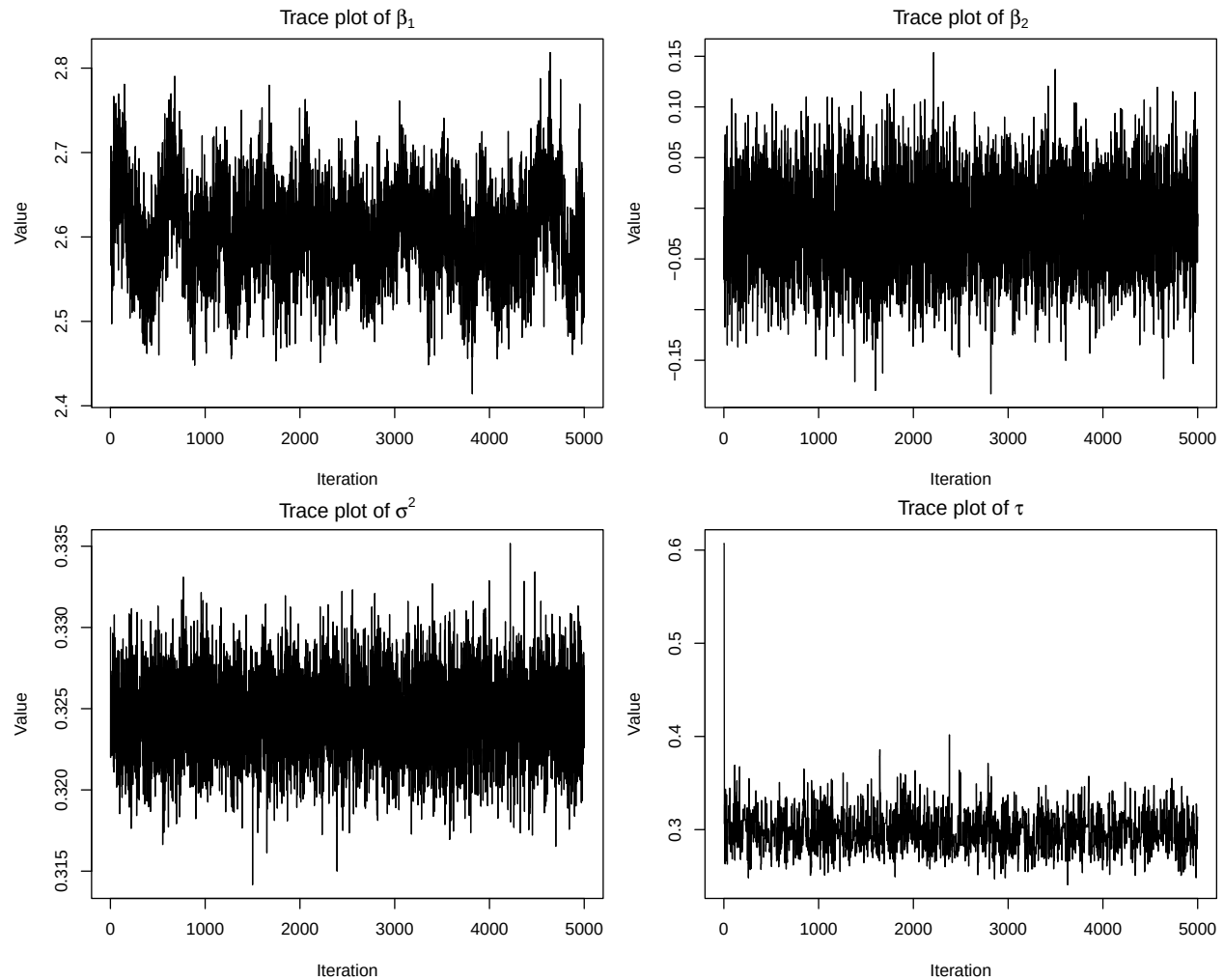
```

xlab = "Iteration", ylab = "Value")

plot(mcmc_fit$sigma2, type = "l",
     main = expression(paste("Trace plot of ", sigma^2)),
     xlab = "Iteration", ylab = "Value")

plot(mcmc_fit$tau, type = "l",
     main = expression(paste("Trace plot of ", tau)),
     xlab = "Iteration", ylab = "Value")

```



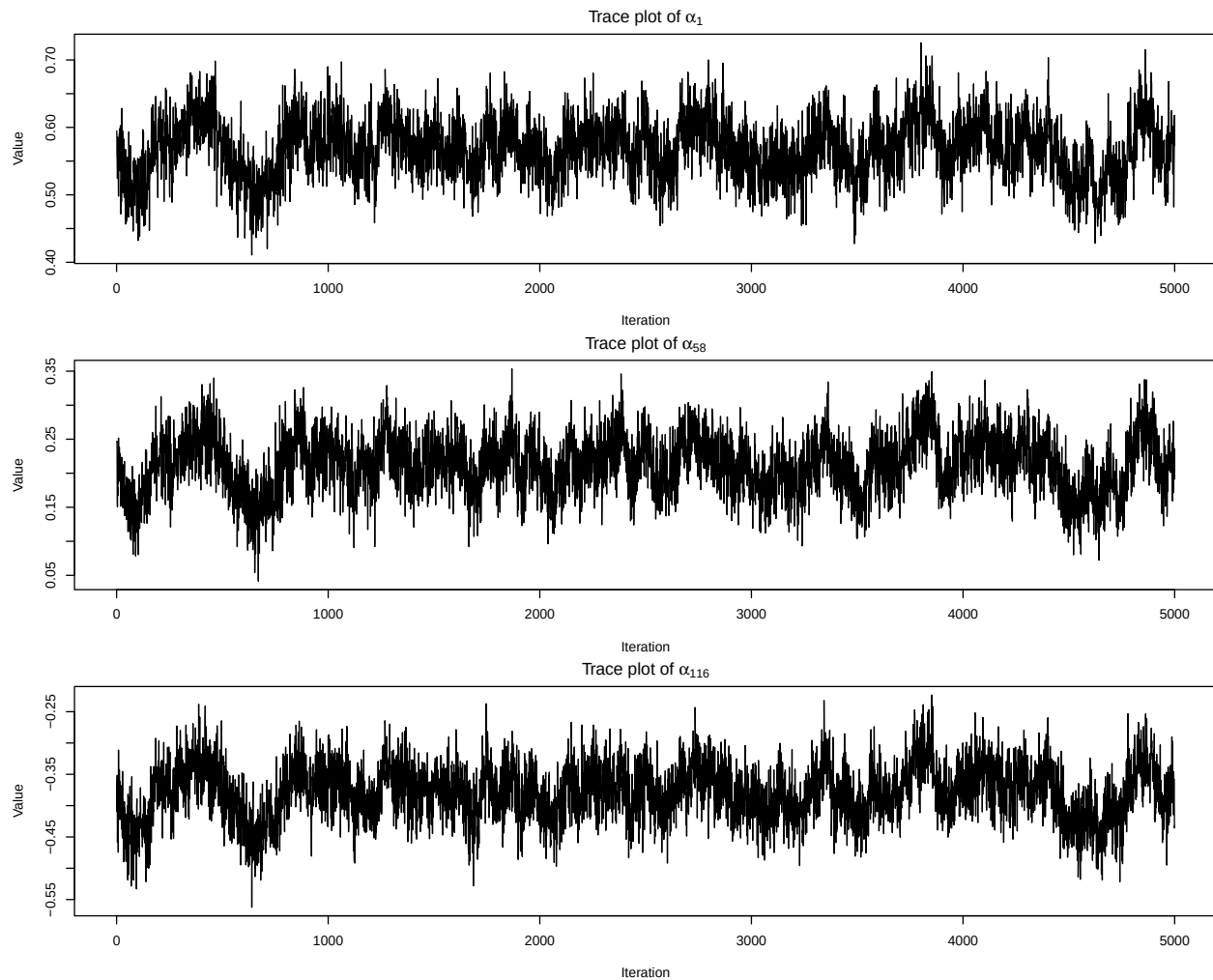
```

J <- ncol(mcmc_fit$alpha)
alpha_idx <- unique(c(1, round(J / 2), J))

par(mfrow = c(length(alpha_idx), 1), mar = c(4, 4, 2, 1))

for (j in alpha_idx) {
  plot(mcmc_fit$alpha[, j], type = "l",
       main = bquote(paste("Trace plot of ", alpha[.(j)])),
       xlab = "Iteration", ylab = "Value")
}

```



9.2 Visualization: ACF

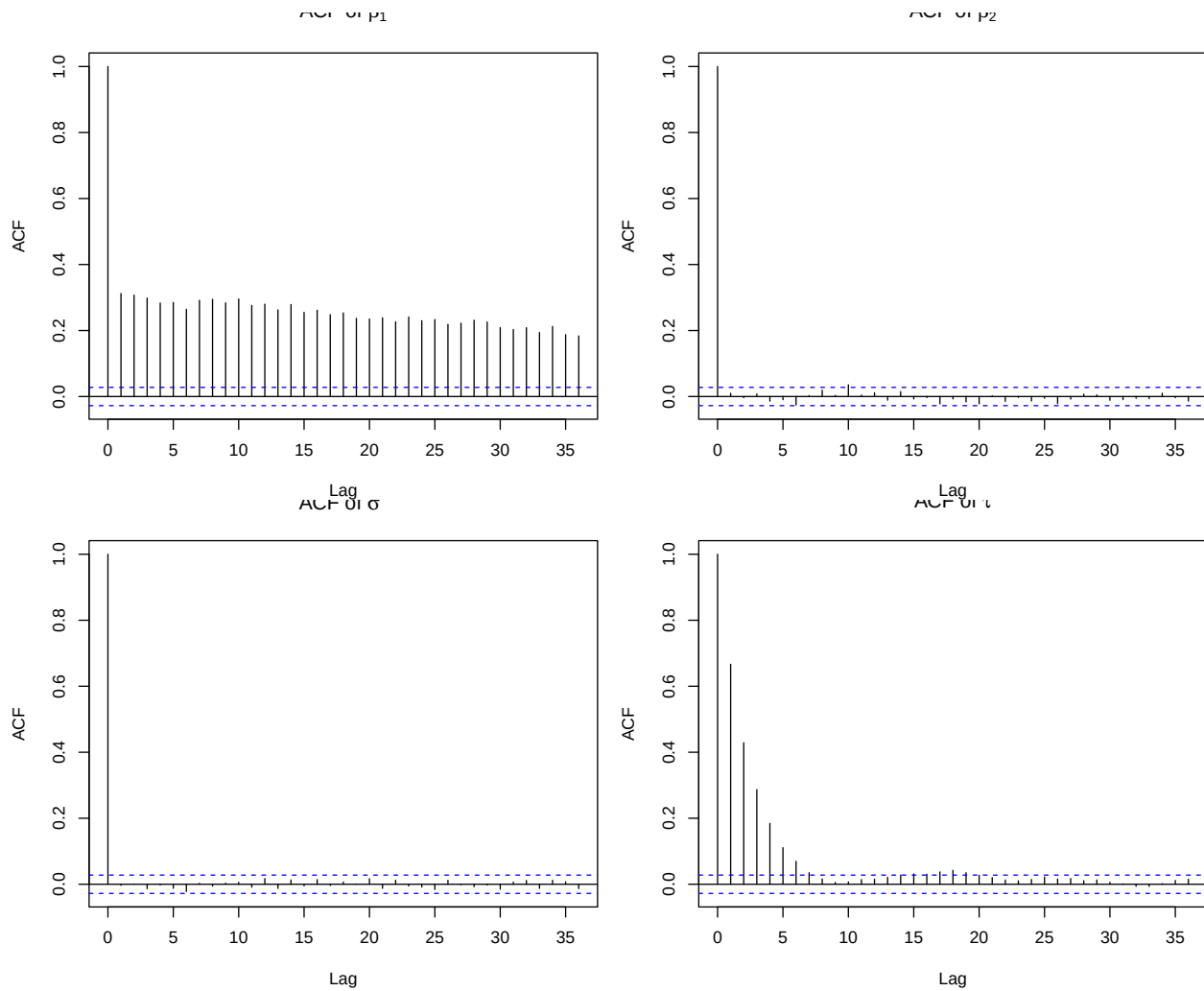
```
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))

acf(mcmc_fit$beta[, 1],
    main = expression(paste("ACF of ", beta[1])))

acf(mcmc_fit$beta[, 2],
    main = expression(paste("ACF of ", beta[2])))

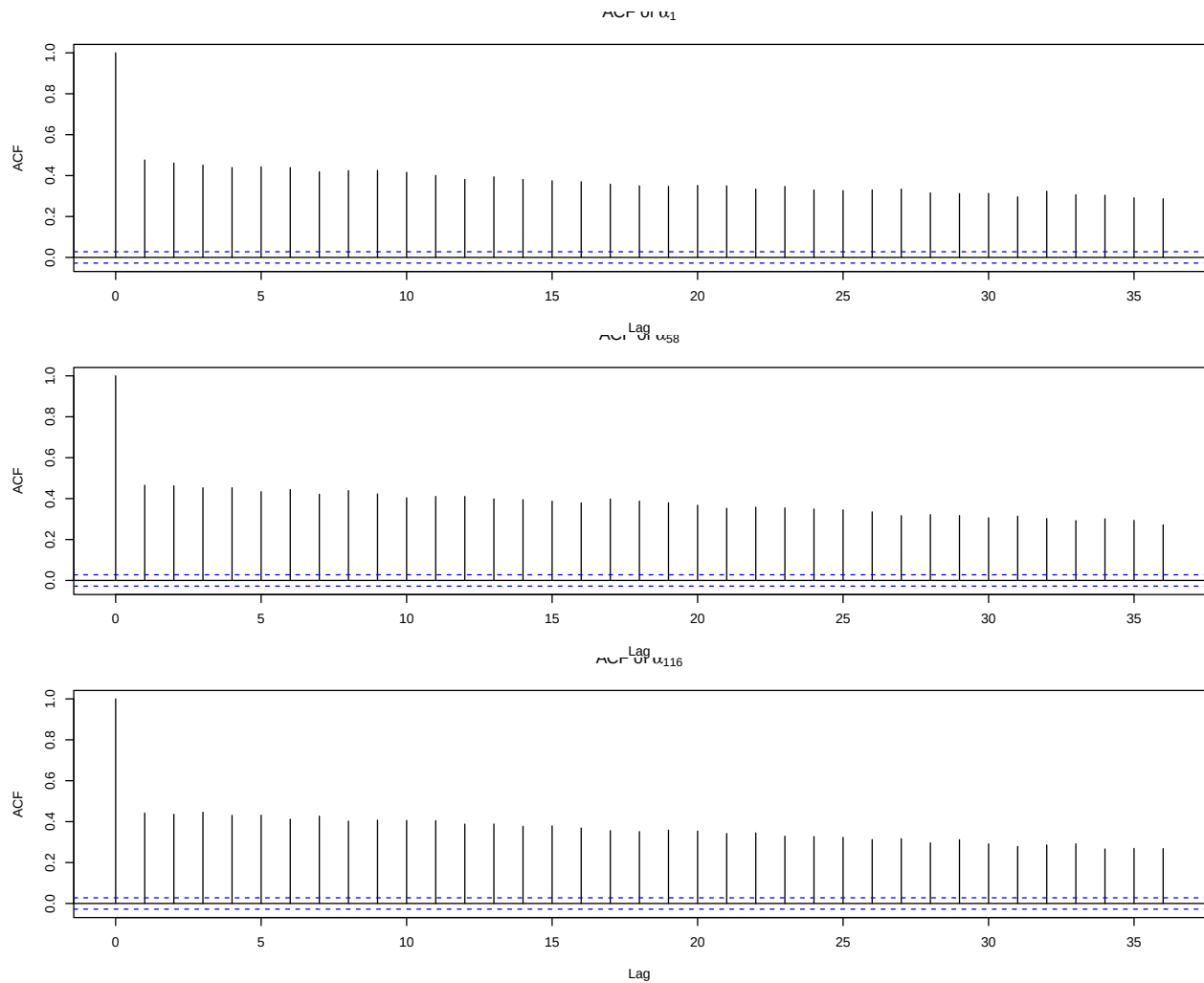
acf(mcmc_fit$sigma2,
    main = expression(paste("ACF of ", sigma^2)))

acf(mcmc_fit$tau,
    main = expression(paste("ACF of ", tau)))
```



```
par(mfrow = c(length(alpha_idx), 1), mar = c(4, 4, 2, 1))

for (j in alpha_idx) {
  acf(mcmc_fit$alpha[, j],
      main = bquote(paste("ACF of ", alpha[.(j)])))
}
```



9.3 Choose burn in iterations

```
burn_in <- 200

post <- post_process_mcmc(mcmc_fit, burn_in = burn_in, thin = 5)

post$accept_rate_tau

## [1] 0.3875
```

9.4 Trace plots post burn-in

```
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))

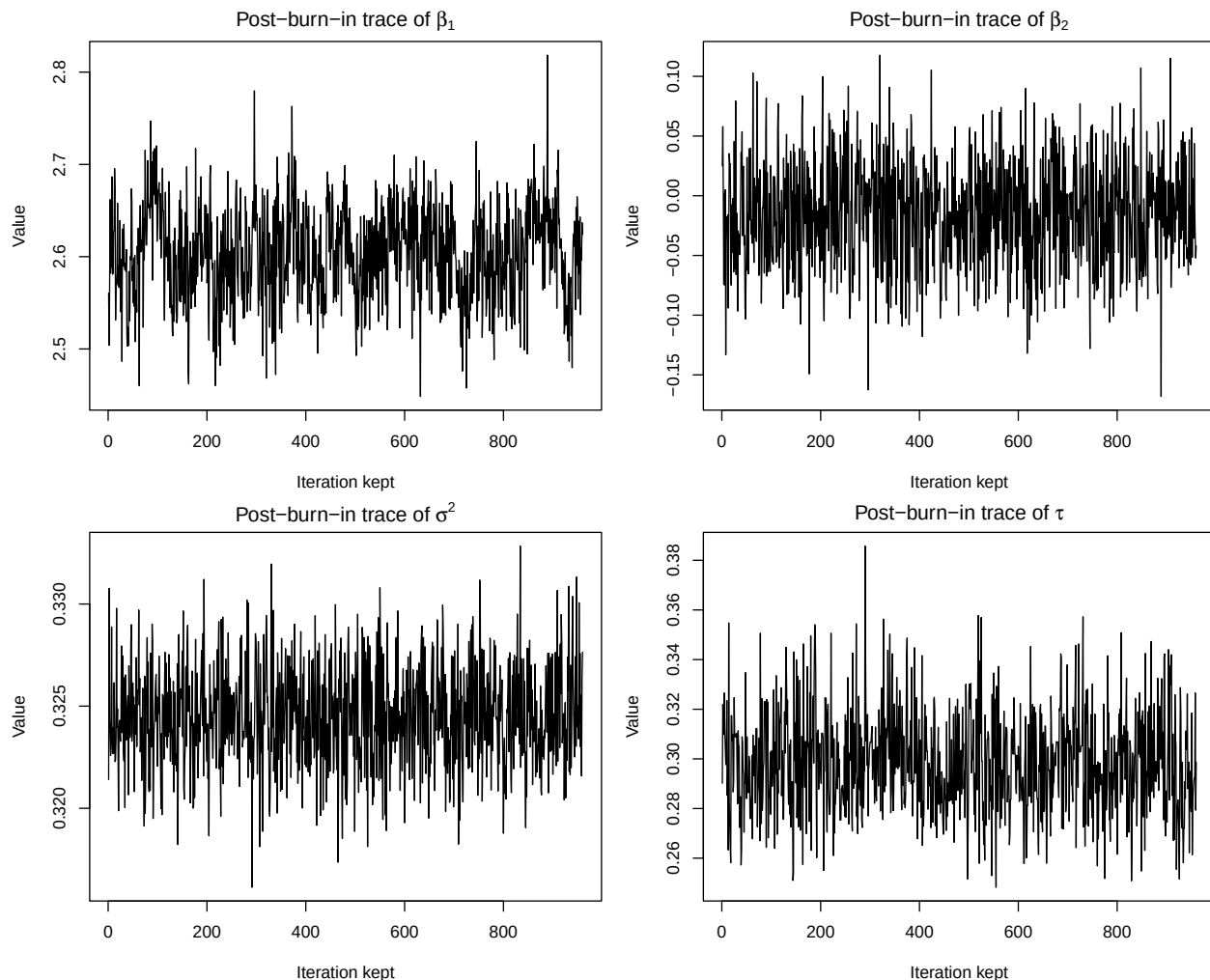
plot(post$post_beta[, 1], type = "l",
     main = expression(paste("Post-burn-in trace of ", beta[1])),
     xlab = "Iteration kept", ylab = "Value")

plot(post$post_beta[, 2], type = "l",
     main = expression(paste("Post-burn-in trace of ", beta[2])),
     xlab = "Iteration kept", ylab = "Value")
```



```
plot(post$post_sigma2, type = "l",
     main = expression(paste("Post-burn-in trace of ", sigma^2)),
     xlab = "Iteration kept", ylab = "Value")

plot(post$post_tau, type = "l",
     main = expression(paste("Post-burn-in trace of ", tau)),
     xlab = "Iteration kept", ylab = "Value")
```



Visual inspection The trace plots of the post-burn-in chains show no evidence of non-stationarity or systematic trends. All parameters fluctuate around stable levels, suggesting that the burn-in period has successfully removed the initial transient phase. The chains exhibit satisfactory mixing, although some residual autocorrelation is visible for selected parameters. Overall, the visual inspection supports convergence and justifies the use of the retained samples for posterior inference.

9.5 ACF post burn-in

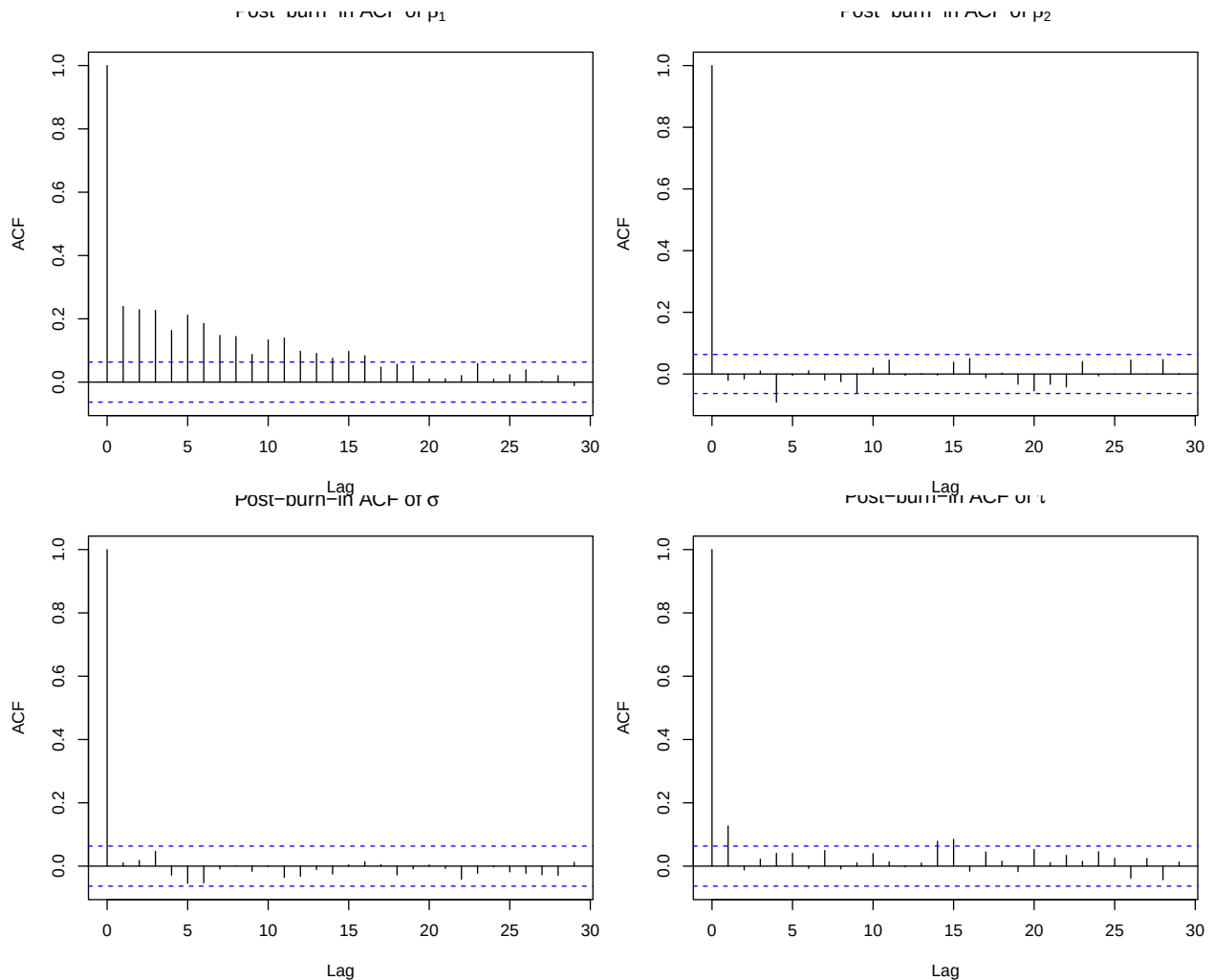
```
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))

acf(post$post_beta[, 1],
    main = expression(paste("Post-burn-in ACF of ", beta[1])))
```

```
acf(post$post_beta[, 2],
    main = expression(paste("Post-burn-in ACF of ", beta[2])))

acf(post$post_sigma2,
    main = expression(paste("Post-burn-in ACF of ", sigma^2)))

acf(post$post_tau,
    main = expression(paste("Post-burn-in ACF of ", tau)))
```



ESS

```
library(coda)
```

```
mcmc_beta <- mcmc(post$post_beta)
mcmc_alpha <- mcmc(post$post_alpha)
mcmc_sigma2 <- mcmc(post$post_sigma2)
mcmc_tau <- mcmc(post$post_tau)
```

```
effectiveSize(mcmc_beta)
```

```
##      var1      var2      var3      var4      var5      var6      var7      var8
## 178.4195 1215.3986 960.0000 960.0000 960.0000 960.0000 960.0000 960.0000
##      var9      var10     var11     var12     var13     var14     var15     var16
```

```
## 960.0000 960.0000 960.0000 960.0000 960.0000 960.0000 960.0000 960.0000
##      var17      var18
## 960.0000 960.0000
```

```
effectiveSize(mcmc_alpha)
```

```
##      var1      var2      var3      var4      var5      var6      var7      var8
## 77.31887 138.04737 85.07854 77.20666 97.62129 127.91207 84.39061 93.07811
##      var9      var10     var11     var12     var13     var14     var15     var16
## 108.82669 86.72049 107.05423 106.25715 100.92763 98.13364 75.13184 115.60673
##      var17     var18     var19     var20     var21     var22     var23     var24
## 87.52595 100.58386 99.83505 86.62592 92.55955 95.81918 83.68467 78.14120
##      var25     var26     var27     var28     var29     var30     var31     var32
## 97.21220 114.79240 100.02824 88.81251 241.63130 65.90789 309.27116 80.16786
##      var33     var34     var35     var36     var37     var38     var39     var40
## 97.80751 121.77930 80.11136 118.88900 109.35149 93.57674 210.54151 239.82155
##      var41     var42     var43     var44     var45     var46     var47     var48
## 263.42533 115.77508 81.18561 76.28422 207.70041 104.56468 104.62707 107.74697
##      var49     var50     var51     var52     var53     var54     var55     var56
## 88.45657 83.99633 82.01833 96.44818 103.91842 331.73363 110.84804 108.31592
##      var57     var58     var59     var60     var61     var62     var63     var64
## 87.54468 76.67296 103.60831 65.89096 107.51303 89.63649 109.25642 81.61830
##      var65     var66     var67     var68     var69     var70     var71     var72
## 93.49420 85.20361 106.48514 118.46933 117.72795 88.41029 125.61047 75.01037
##      var73     var74     var75     var76     var77     var78     var79     var80
## 95.94259 86.44177 79.86536 76.01577 136.52852 77.61638 116.65849 86.07406
##      var81     var82     var83     var84     var85     var86     var87     var88
## 87.66997 110.56194 122.92568 89.15714 88.41709 109.27010 113.59730 96.53816
##      var89     var90     var91     var92     var93     var94     var95     var96
## 70.47446 105.27801 90.69108 250.11348 341.11590 287.82932 159.20758 352.85200
##      var97     var98     var99     var100    var101    var102    var103    var104
## 97.68549 86.62640 93.95693 122.21882 91.00623 265.83802 88.31744 86.13802
##      var105    var106    var107    var108    var109    var110    var111    var112
## 182.10134 122.40150 272.04181 85.76792 72.14616 100.25281 109.15425 85.26331
##      var113    var114    var115    var116
## 103.43023 178.07090 79.15976 112.36994
```

```
effectiveSize(mcmc_sigma2)
```

```
## var1
## 960
```

```
effectiveSize(mcmc_tau)
```

```
##      var1
## 742.9954
```

Due to autocorrelation in the MCMC draws, the effective sample size (ESS) is used to assess the amount of independent information in the chain. The ESS values are sufficiently large, indicating good mixing of the sampler.

10 Posterior summaries

```
posterior_summary <- function(x) {
  c(
    mean = mean(x),
```

```

    sd = sd(x),
    q2.5 = quantile(x, 0.025),
    median = median(x),
    q97.5 = quantile(x, 0.975)
  )
}

beta_summary <- t(apply(post$post_beta, 2, posterior_summary))
sigma2_summary <- posterior_summary(post$post_sigma2)
tau_summary <- posterior_summary(post$post_tau)

beta_summary

##           mean          sd    q2.5.2.5%      median  q97.5.97.5%
## [1,]  2.603418081 0.05227638  2.503088301  2.604618751  2.698884667
## [2,] -0.015749356 0.04485902 -0.098919389 -0.014005639  0.069808638
## [3,] -0.362021522 0.04479207 -0.445499340 -0.361363611 -0.274977980
## [4,] -0.614974779 0.04461996 -0.697072516 -0.614183738 -0.528397762
## [5,] -0.741661508 0.04475021 -0.826890367 -0.741035716 -0.657262894
## [6,] -0.119440301 0.04482782 -0.202748396 -0.117612860 -0.031183765
## [7,] -0.597696184 0.04499852 -0.681118148 -0.597782345 -0.506304894
## [8,] -0.425342330 0.04502280 -0.507909098 -0.425383033 -0.341742110
## [9,] -0.799916326 0.04538712 -0.884326920 -0.799863960 -0.709958316
## [10,] -0.605106712 0.04494935 -0.689595761 -0.603978984 -0.518062816
## [11,] -0.380865249 0.04504541 -0.466546496 -0.381069863 -0.291663007
## [12,] -0.067539893 0.04518908 -0.150266467 -0.066730800  0.019025060
## [13,]  0.010123663 0.01190534 -0.012221034  0.009866626  0.034299241
## [14,]  0.007590409 0.01218215 -0.015439528  0.007740871  0.031843365
## [15,]  0.001124320 0.01233144 -0.022907245  0.001650826  0.025221625
## [16,]  0.012202543 0.01179446 -0.009870504  0.012029936  0.034668027
## [17,]  0.024737899 0.01200304  0.001580576  0.024947168  0.047981173
## [18,] -0.017148195 0.01187840 -0.040633864 -0.017234633  0.005620652

sigma2_summary

##           mean          sd    q2.5.2.5%      median  q97.5.97.5%
## 0.324508332 0.002530976 0.319686983 0.324396601 0.329439915

tau_summary

##           mean          sd    q2.5.2.5%      median  q97.5.97.5%
## 0.29850692  0.02047837  0.26102340  0.29699114  0.34312950

```

Inspection of the trace plots suggested only a short transient phase, so the first 200 iterations were discarded as burn-in. Residual autocorrelation remained for some parameters, especially the intercept and selected random effects; therefore, a mild thinning scheme (every 5th draw) was applied. The retained chains appeared stable, and the acceptance rate for the Metropolis-Hastings update of τ was satisfactory.

11 Sensitivity analysis

This section presents a simple sensitivity analysis to assess the robustness of posterior inference to prior choices. In particular, I consider:

- $c \in \{10, 100, 1000\}$ for the prior covariance $B_0 = cI$
- $A \in \{2.5, 5, 10\}$ for the half-Cauchy prior on τ
- $b_\sigma \in \{0.5, 1\}$ for the inverse-gamma prior on σ^2

```

run_one_spec <- function(c_prior, A_tau, b_sigma,
                        scenario = NA_character_,
                        n_iter = 5000,
                        burn_in = 200,
                        thin = 5,
                        seed = 123) {

  fit <- run_mwg_sampler(
    y_tilde = y_tilde,
    X = X,
    station_index = station_index,
    n_j = n_j,
    b0 = rep(0, ncol(X)),
    B0 = diag(c_prior, ncol(X)),
    a_sigma = 2,
    b_sigma = b_sigma,
    A_tau = A_tau,
    n_iter = n_iter,
    proposal_sd = 0.2,
    seed = seed
  )

  post <- post_process_mcmc(
    mcmc_fit = fit,
    burn_in = burn_in,
    thin = thin
  )

  data.frame(
    scenario = scenario,
    c_prior = c_prior,
    A_tau = A_tau,
    b_sigma = b_sigma,
    beta_intercept_mean = mean(post$post_beta[, 1]),
    sigma2_mean = mean(post$post_sigma2),
    tau_mean = mean(post$post_tau),
    tau_accept_post = post$accept_rate_tau
  )
}

```

11.1 One-at-a-time sensitivity analysis

```

baseline_row <- data.frame(
  scenario = "Baseline",
  c_prior = 100,
  A_tau = 5,
  b_sigma = 0.5,
  beta_intercept_mean = mean(post$post_beta[, 1]),
  sigma2_mean = mean(post$post_sigma2),
  tau_mean = mean(post$post_tau),
  tau_accept_post = post$accept_rate_tau
)

```

```
sensitivity_results <- bind_rows(
  baseline_row,
  run_one_spec(10, 5, 0.5, scenario = "B0 smaller"),
  run_one_spec(1000, 5, 0.5, scenario = "B0 larger"),
  run_one_spec(100, 2.5, 0.5, scenario = "A_tau smaller"),
  run_one_spec(100, 10, 0.5, scenario = "A_tau larger"),
  run_one_spec(100, 5, 1, scenario = "b_sigma larger")
)

sensitivity_results
```

##	scenario	c_prior	A_tau	b_sigma	beta_intercept_mean	sigma2_mean
## 1	Baseline	100	5.0	0.5	2.603418	0.3245083
## 2	B0 smaller	10	5.0	0.5	2.601836	0.3245083
## 3	B0 larger	1000	5.0	0.5	2.603480	0.3245082
## 4	A_tau smaller	100	2.5	0.5	2.603325	0.3245082
## 5	A_tau larger	100	10.0	0.5	2.603418	0.3245083
## 6	b_sigma larger	100	5.0	1.0	2.603363	0.3245404

##	tau_mean	tau_accept_post
## 1	0.2985069	0.3875000
## 2	0.2983755	0.3937500
## 3	0.2986517	0.4010417
## 4	0.2988245	0.3843750
## 5	0.2985069	0.3875000
## 6	0.2981771	0.3927083

The sensitivity analysis shows that posterior estimates are remarkably stable across a wide range of prior specifications. In particular, the posterior mean of the intercept, the residual variance σ^2 , and the hyperparameter τ remain virtually unchanged when varying the prior scale c , the hyperparameter A , and the inverse-gamma parameter b_σ . As expected, the hyperparameter τ shows slightly higher sensitivity compared to the regression coefficients, although the magnitude of variation remains negligible. This suggests that the data are highly informative and that the posterior distribution is largely driven by the likelihood rather than the prior. The results are therefore robust to reasonable prior choices.

12 CODA: Geweke Diagnostics

```
library(coda)

mcmc_beta1 <- mcmc(post$post_beta[, 1])
mcmc_beta2 <- mcmc(post$post_beta[, 2])
mcmc_sigma2 <- mcmc(post$post_sigma2)
mcmc_tau <- mcmc(post$post_tau)

geweke_results <- data.frame(
  parameter = c("beta_1", "beta_2", "sigma2", "tau"),
  z_score = c(
    geweke.diag(mcmc_beta1)$z,
    geweke.diag(mcmc_beta2)$z,
    geweke.diag(mcmc_sigma2)$z,
    geweke.diag(mcmc_tau)$z
  )
)
```

```
geweke_results
```

```
##   parameter    z_score
## 1   beta_1    0.2233956
## 2   beta_2   -0.7608000
## 3   sigma2   -0.6490586
## 4     tau   -0.2004884
```

```
J <- ncol(post$post_alpha)
alpha_idx <- unique(c(1, round(J / 2), J))
```

```
alpha_geweke <- data.frame(
  parameter = paste0("alpha_", alpha_idx),
  z_score = sapply(alpha_idx, function(j) geweke.diag(mcmc(post$post_alpha[, j]))$z)
)
```

```
alpha_geweke
```

```
##   parameter    z_score
## 1   alpha_1    0.11113809
## 2   alpha_58  -0.03092227
## 3 alpha_116    0.06347014
```

The Geweke diagnostic provides no evidence against convergence. All z-scores are well below the conventional threshold of 1.96 in absolute value, both for the main model parameters and for the selected random effects. This finding is consistent with the visual inspection of the trace plots and supports the use of the retained draws for posterior inference.

13 Posterior Predictive Checks

```
# Number of posterior draws to use for PPC
S <- min(100, nrow(post$post_beta))

set.seed(123)

# Randomly select posterior draws
draw_ids <- sample(seq_len(nrow(post$post_beta)), S)

# Matrix to store replicated datasets
yrep_mat <- matrix(NA_real_, nrow = length(y_tilde), ncol = S)

for (s in seq_along(draw_ids)) {
  d <- draw_ids[s]

  beta_s <- post$post_beta[d, ]
  alpha_s <- post$post_alpha[d, ]
  sigma2_s <- post$post_sigma2[d]

  mu_s <- as.vector(X %*% beta_s) + alpha_s[station_index]

  yrep_mat[, s] <- rnorm(
    n = length(y_tilde),
    mean = mu_s,
    sd = sqrt(sigma2_s)
  )
}
```

```

)
}

obs_mean <- mean(y_tilde)
obs_sd   <- sd(y_tilde)

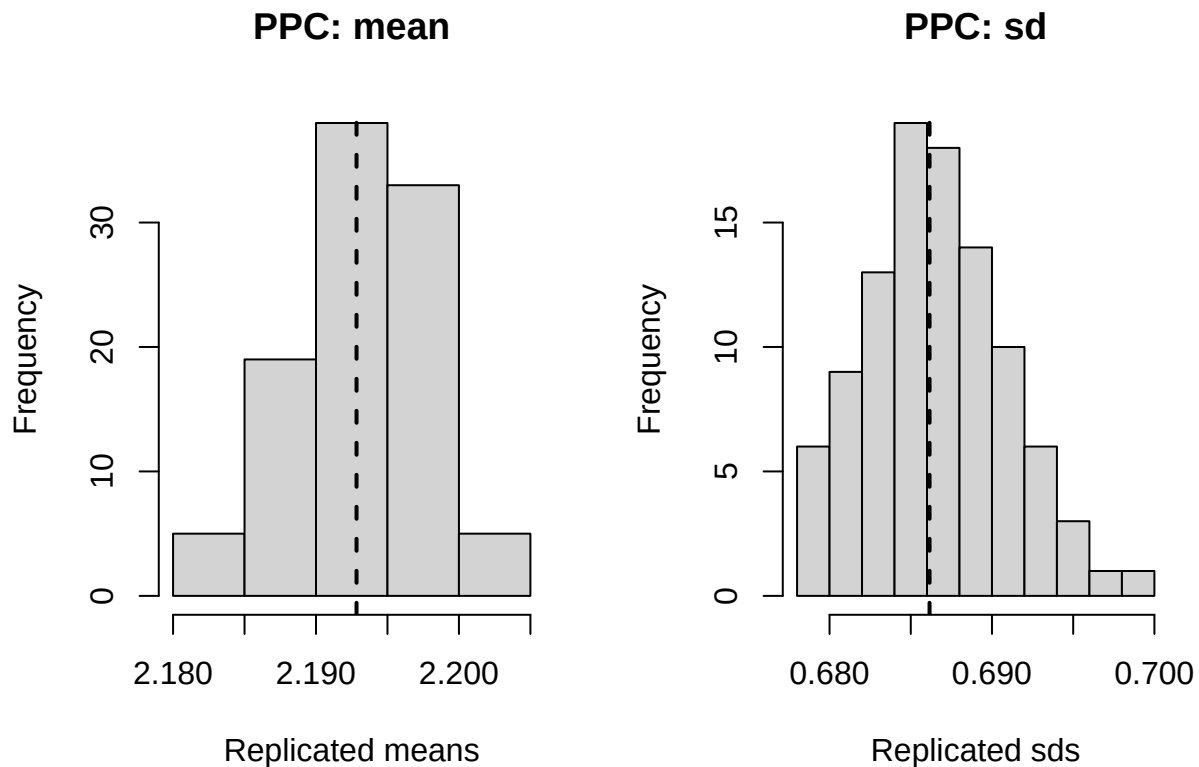
rep_means <- apply(yrep_mat, 2, mean)
rep_sds   <- apply(yrep_mat, 2, sd)

par(mfrow = c(1, 2))

hist(rep_means, main = "PPC: mean", xlab = "Replicated means")
abline(v = obs_mean, lwd = 2, lty = 2)

hist(rep_sds, main = "PPC: sd", xlab = "Replicated sds")
abline(v = obs_sd, lwd = 2, lty = 2)

```



```

par(mfrow = c(1, 1))

plot(density(y_tilde),
     lwd = 2,
     main = "Posterior Predictive Check",
     xlab = "log(PM2.5)",
     ylim = c(0, max(density(y_tilde)$y) * 1.2))

for (s in 1:S) {
  lines(density(yrep_mat[, s]), col = rgb(1, 0, 0, 0.08))
}

lines(density(y_tilde), lwd = 2)

```

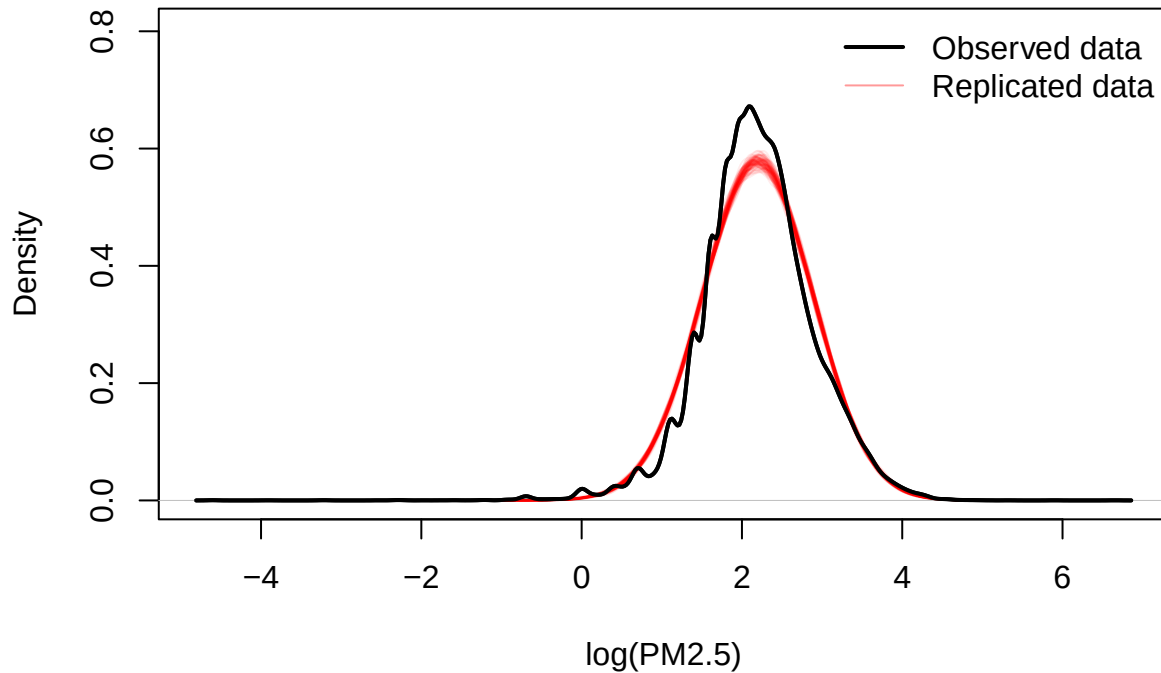


```

legend("topright",
      legend = c("Observed data", "Replicated data"),
      lwd = c(2, 1),
      col = c("black", rgb(1, 0, 0, 0.4)),
      bty = "n")

```

Posterior Predictive Check



The posterior predictive checks indicate a satisfactory model fit. The distribution of replicated means and standard deviations closely matches the observed values, suggesting that the model accurately captures the first two moments of the data.

Furthermore, the posterior predictive density aligns well with the empirical distribution of $\log(\text{PM}_{2.5})$, with only minor discrepancies in the peak and tail regions. Overall, these results suggest that the model provides an adequate representation of the data-generating process, with no major signs of systematic misfit.